Aeronautics and Space Technology Club

IIT (ISM) Dhanbad

Signal Processing Ladder: A Hands-On Problem Series from Sensors to Machine Learning

Problem Statement 14 · Part D: Digital Signal Processing Level 2

Track: Finite Impulse Response Filters, Linear Phase, Window and Equiripple Design, Fast Filtering

Prerequisites: Problems 06, 12 · **Estimated effort:** 12–16 hours

FIR Filter Design: Always Stable, Perfectly Linear Phase, and Never Free

Design low-pass, band-pass, differentiating and Hilbert-transform FIR filters with the window method and with equiripple optimisation, predict the number of taps from the specification, filter long records with the FFT, quantise the coefficients for a fixed-point processor, and build a step counter from a smartphone accelerometer.

General Instructions

- **Thorough Review:** Read the whole problem statement, including the theory, before writing any code. Do the short analytic derivations *by hand first*; the code is there to confirm and extend your understanding, not to replace it.
- **Pacing:** This is a self-paced problem. The split below is a guide for the effort estimate in the header.

Stage	Phase	What you should have at the end of it
1	Think (∼20%)	Theory understood; every analytic result derived; signals, units and sampling rates written down.
2	Build (∼30%)	Core code working end to end on the baseline data and validated against the analytic results.
3	Experiment (∼30%)	Full sweeps, stress tests and the real or synthetic datasets processed; first plots and numbers.
4	Refine (∼20%)	Weaknesses fixed, honest analysis and memo written, repository folder clean.

- **Compute and Tools:** Everything is designed to run on a laptop CPU in Python 3.10+ (NumPy, SciPy, Matplotlib, and where stated pandas, Astropy, scikit-learn, PyWavelets and PyTorch). No GPU and no hardware are required; where a problem offers an optional hardware experiment it is marked as a bonus. A full run should never need more than a few minutes; if yours does, vectorise before you scale up.
- **Data:** Each problem lists (i) a *synthetic generator* from `sp_common.synth` (offline, seeded, with known ground truth) and (ii) *real datasets* to download yourself. Develop and validate on the synthetic data first, then repeat the key result on real data. Never commit large raw data files to the repository.
- **Submission Format:** Push your work to `solutions/14_fir_filter_design/` in the repository: (i) clean, commented, reproducible Jupyter notebook(s) with fixed random seeds, (ii) a short written report (`report.pdf` or `report.md`) containing the requested analysis sections, and (iii)

all figures in a `figures/` sub-folder. The report must be understandable without running the code.

- **Originality:** All code, analysis and written explanations must be your own work. Any external library, snippet, paper or AI tool you use must be cited in the report. You should be able to explain every line you submit.
- **Honesty over Hype:** Validate against analytic results *before* trusting a number. A modest or negative result that is correctly analysed is worth more than an inflated claim. Report error bars, false-alarm rates and failure cases, not only your best run.
- **Units:** Write the units of every quantity and state in the first notebook cell whether a frequency is in Hz or rad/s and whether it is normalised to the sampling rate. Most mistakes in signal processing are factors of 2π, factors of 2 in one-sided spectra, or an off-by-one in the frequency axis.
- **Getting Stuck:** Re-read the theory section, check the self-check targets, then ask your mentor or open an issue in the repository. Describe what you expected, what you got, and what you already tried.

Background and Motivation

A finite impulse response filter computes each output as a weighted sum of the most recent M inputs. It is unconditionally stable (no feedback), can have exactly linear phase (a pure delay, with no waveform distortion) and is easy to analyse. The price is length: a sharp transition needs tens or hundreds of taps, and those taps cost multiplications and delay.

The design question is always the same: given a passband edge, stopband edge, ripple and attenuation, how many taps are needed and what are their values? You will answer it with analytic window methods, with Kaiser's design formulas, and with the optimal Parks–McClellan algorithm, then use the resulting filters on real data (counting steps from walking acceleration) and on the model of a flight-control sensor where the filter's group delay is part of the control loop.

Theoretical Framework

1. FIR Filters and Linear Phase

$$y[n] = \sum_{k=0}^{M-1} h[k]\, x[n-k], \qquad H(\mathrm{e}^{\mathrm{j}\omega}) = \sum_{k=0}^{M-1} h[k]\mathrm{e}^{-\mathrm{j}\omega k}. \tag{1}$$

If $h[k] = \pm h[M-1-k]$ the phase is linear and the group delay is the constant $(M-1)/2$ samples. Four types exist: I (symmetric, odd M), II (symmetric, even M; forced zero at $z=-1$, so no high-pass), III (antisymmetric, odd M; zeros at $z=\pm 1$: differentiators and Hilbert transformers) and IV (antisymmetric, even M; zero at $z=1$).

2. Window Method

Truncate the ideal low-pass impulse response $h_d[n] = \sin(\omega_c(n-\alpha))/(\pi(n-\alpha))$, $\alpha = (M-1)/2$, with a window $w[n]$: $h[n] = h_d[n]w[n]$. The window fixes the stopband attenuation (≈ 21 dB rectangular, 44 Hann, 53 Hamming, 74 Blackman, measured at the first stopband ripple) and the transition width $\Delta f \approx \{0.9, 3.1, 3.3, 5.5\}/M \cdot f_s$.

3. Kaiser Design

For an attenuation A dB and a transition width $\Delta\omega$ (rad/sample),

$$M - 1 \approx \frac{A - 7.95}{2.285\,\Delta\omega}, \qquad \beta = \begin{cases} 0.1102(A - 8.7) & A > 50 \\ 0.5842(A - 21)^{0.4} + 0.07886(A - 21) & 21 \le A \le 50 \\ 0 & A < 21 \end{cases} \tag{2}$$

and $w[n] = I_0\big(\beta\sqrt{1 - (2n/(M-1) - 1)^2}\big)/I_0(\beta)$.

4. Equiripple (Parks–McClellan)

The Remez exchange algorithm finds the type-I filter that minimises the maximum weighted error over the pass and stop bands; by the alternation theorem the optimal error curve equioscillates at least $r+2$ times. A common order estimate (Kaiser/Herrmann) is $M \approx \big(-10\log_{10}(\delta_p\delta_s) - 13\big)/(14.6\,\Delta f/f_s) + 1$.

5. Fast Filtering

Direct filtering costs M multiplications per sample. For long filters, overlap-add with FFT blocks of size $N_{\text{fft}} \ge L + M - 1$ costs $\mathcal{O}(\log N_{\text{fft}})$ per sample.

6. Coefficient Quantisation

Rounding the coefficients to b bits perturbs the response; the stopband floor rises to about $-6\,b$ dB relative to the passband.

Data and Simulation Specification

Specification A	$f_s = 1$ kHz, passband edge 100 Hz, stopband edge 200 Hz, $A = 60$ dB stopband attenuation, passband ripple < 0.1 dB.
Windowed sinc	Cutoff 100 Hz, $f_s = 1$ kHz, $M = 101$ taps with each of rectangular, Hann, Hamming, Blackman windows.
Type tests	One filter of each linear-phase type; differentiator and Hilbert transformer with $M = 31$ and $M = 32$.
Long record	10^6 samples of noise through a 1001-tap filter; block sizes 2^8–2^{16}.
Pedometer (real data)	Phyphox accelerometer log of about 60 s of walking with the phone in a trouser pocket or in the hand (≈ 100 samples per second): use the magnitude $\lvert\mathbf{a}\rvert$, count your steps by hand as ground truth. Gait band 0.5–3 Hz.
Vibration (synthetic)	`sp_common.synth.bearing_vibration(fault="none")` plus an added low-frequency drift; use the IMU example with $f_s = 1$ kHz, wanted band to 50 Hz.

Your Task

1. **Windowed-sinc design (Stage 2).** Implement the windowed-sinc low-pass from scratch for the four windows. Plot magnitude (dB) and phase, verify the constant group delay $(M-1)/2$, and measure transition width and peak stopband attenuation of each; compare with the figures in the theory section.
2. **Kaiser design.** Design Specification A with Eq. (2): compute M and β by hand, build the filter, and check it against the specification with a dense frequency grid; compare with `scipy.signal.kaiserord` and `firwin`. Report how often the first attempt fails and what you adjust.

3. **Equiripple design.** Design Specification A with `scipy.signal.remez`, find the smallest M that meets $A = 60\,\text{dB}$ by bisection, plot the equiripple error and compare the number of taps and the maximum ripple with the Kaiser design. Explore the effect of the band weights.
4. **Linear-phase types.** Build one filter of each type I–IV, show their zero patterns and the forced zeros at $z = \pm 1$ and explain which types can realise a high-pass or a band-stop. Design a differentiator and a Hilbert transformer, test them on sinusoids (the differentiator gain must be ω, the Hilbert transformer must shift phase by $-90°$) and compute the envelope of an AM signal.
5. **Fast filtering.** Implement overlap-add FIR filtering with FFT blocks and verify against `lfilter` on the long record. Plot run time versus block size and versus M; find the crossover with direct filtering. Compensate the group delay so that input and output align.
6. **Pedometer.** Design a band-pass FIR (Kaiser) for the gait band, apply it to your accelerometer magnitude, detect steps by peak picking with a minimum spacing, and compare with your hand count. Study how the number of taps and the band edges change delay, missed steps and false steps.
7. **Coefficient quantisation.** Quantise your Specification-A filter to 16, 12, 10 and 8 bits and plot the stopband degradation. Implement a bit-exact integer FIR (32-bit accumulator, saturation) and compare with floating point on a test signal; state the number of guard bits needed to avoid overflow for the filter's $\sum |h|$.

Self-check targets

Specification A: the Kaiser formula gives $M - 1 \approx 36.3$ (so 38 taps; `kaiserord` returns 38) with $\beta = 5.653$; equiripple needs about 34–35 taps for the same 60 dB. A type-I filter of M taps has group delay $(M-1)/2$ samples (here 19 ms for $M = 39$ at $f_s = 1\,\text{kHz}$). Windowed-sinc stopband attenuations are about 21, 44, 53, 74 dB for rectangular, Hann, Hamming, Blackman.

Aerospace Application Link

A 1 kHz accelerometer feeds a flight-control loop with a closed-loop crossover near 5 Hz. The FIR filter of Specification A has $(M-1)/2 = 19$ samples of delay (19 ms), a phase lag of $360 \times 5 \times 0.019 = 34°$ at crossover. Compute the phase margin you lose for each filter you designed (Kaiser, equiripple, a minimum-length filter you design with a relaxed specification), and recommend a design that keeps the phase loss below $10°$ while attenuating the 150 Hz structural vibration by at least 40 dB. State what an IIR alternative (problem 15) would buy and what it would cost.

Deliverables

D1 **Windowed-sinc notebook:** four designs with measured transition widths and attenuations versus the theory table.

D2 **Kaiser and equiripple comparison:** order calculations, specification checks, tap-count comparison.

D3 **Linear-phase types:** zero plots, the differentiator and Hilbert transformer tests.

D4 **Fast filtering and quantisation:** the overlap-add implementation with timing, the coefficient-quantisation and fixed-point results.

D5 **Pedometer:** designed filter, detected steps, hand count and error analysis.

D6 **Memo (250 words):** "Which filter for the control-loop accelerometer, and why?" Cite at

least two external sources.

Bonus (a) Implement the Remez exchange algorithm yourself and compare with SciPy. (b) Design a minimum-phase FIR from your linear-phase design by spectral factorisation and compare group delays. (c) Implement a polyphase FIR on a microcontroller.

Learning Resources

FIR Design

- Oppenheim, A.V., Schafer, R.W. *Discrete-Time Signal Processing*, Ch. 7 (filter design techniques: windowing, Kaiser, Parks–McClellan).
- Parks, T.W., McClellan, J.H. (1972). "Chebyshev approximation for nonrecursive digital filters with linear phase." IEEE Trans. Circuit Theory 19, 189.
- Kaiser, J.F. (1974). "Nonrecursive digital filter design using the I_0-sinh window function." Proc. IEEE Int. Symp. Circuits and Systems, 20.
- Proakis, J.G., Manolakis, D.G. *Digital Signal Processing*, Ch. 10 (design of digital filters).

Practice

- Lyons, R.G. *Understanding Digital Signal Processing*, Ch. 5 (FIR filters).
- Smith, S.W. *The Scientist and Engineer's Guide to Digital Signal Processing*, Ch. 14–18 (windowed-sinc filters, FFT convolution); `https://www.dspguide.com`.

Tools

- SciPy `signal.firwin`, `signal.kaiserord`, `signal.remez`, `signal.oaconvolve`: `https://docs.scipy.org/doc/scipy/reference/signal.html`; Phyphox: `https://phyphox.org`

Evaluation

Score your own submission out of 100 on five equally weighted criteria, then ask your mentor to score it independently and compare.

Criterion	What to look for	Marks
Conceptual depth	Correct mathematics and physics; every result explained through a mechanism (why the spectrum looks like this), not only through code behaviour.	/20
Implementation quality	Correct, modular, commented, reproducible code; no unexplained magic numbers; analytic checks and self-check targets pass.	/20
Aerospace-application integration	The signal-processing result is carried through to a measurable consequence for a sensor, a link, a flight system or a detection decision, as the task requires.	/20
Analysis and critical thinking	Error bars, convergence, comparison to theory, honest treatment of limitations, false alarms and failure cases.	/20
Communication	Clear report, labelled plots (units, legends, captions) and a memo that a non-expert student could follow.	/20

Bonus tasks are not part of the 100 marks. A bonus attempted badly does not cost you marks, and a well-executed one is the best sign that you own the material.